Report

On

CLASSROOM AND DAILY-USE OBJECT RECOGNITION USING YOLOv8

Bachelor of Technology (B.Tech)

in

Computer Science (SCOPE)

Submitted By	[Your Full Name]
Enrollment No.	[Your Enrollment Number]
Branch / Year	CSE CORE (2ND SEM)
Session	Winter Semester
Guide / Faculty	Rakesh Srivastava

VIT BHOPAL UNIVERSITY

Submission Date: 31st March 2025

Acknowledgement

I would like to express my sincere gratitude to Rakesh Srivastava, faculty of the SCAI, for providing constant guidance, motivation, and support throughout this case study project. Without his/her valuable suggestions and encouragement, the completion of this work would not have been possible.

I am also thankful to the Head of Department and all the faculty members of the AI & ML department for their continuous cooperation and for providing the necessary laboratory facilities. I am grateful to my friends and classmates for their help and constructive feedback during the development and testing phases of this project.

I also acknowledge the open-source community, specifically the developers of Ultralytics YOLOv8, OpenCV, and the contributors of Roboflow and Kaggle platforms, from where the dataset was obtained free of cost for academic and educational purposes.

Debayan Mitra
Enrolment No.: 25BCE10801
B. Tech | 1st Year | CSE core

Abstract

Object detection is one of the most significant and actively researched areas in the field of computer vision and deep learning. It involves identifying the presence and location of objects within a digital image or video frame. In recent years, YOLO (You Only Look Once) based architectures have become the industry standard for real-time object detection tasks due to their high speed and competitive accuracy. This case study presents the complete design, development, training, and deployment of an object detection system using YOLOv8 Nano, the latest and most optimized iteration of the YOLO family developed by Ultralytics.

The objective of this project was to train a lightweight deep learning model capable of detecting classroom and daily-use objects such as bottles, chairs, books, laptops, and cups. The model was trained on a custom YOLO-formatted dataset for 10 epochs on a local system, without the requirement of any external cloud computing service. Post training, the best-performing model checkpoint (best.pt) was deployed for real-time object detection using a webcam and the OpenCV library. The system was successfully able to detect and locate objects with visible bounding boxes and label annotations.

This project demonstrates that even first-year undergraduate students can implement, train, and deploy a functional AI-based computer vision system using freely available tools and a standard laptop. The entire pipeline from dataset preparation to real-time inference is documented in detail in this report.

Keywords: *Object Detection, YOLOv8, Computer Vision, Deep Learning, OpenCV, Real-Time Detection, Machine Learning, bounding box, mAP, IoU.*

Table of Contents

Acknowledgement	1
Abstract	2
Chapter 1: Introduction	3
Chapter 2: Problem Statement and Objectives	4
Chapter 3: Literature Review / Background Study	5
Chapter 4: Dataset Description and Preprocessing	6
Chapter 5: Model Architecture and Methodology	7
Chapter 6: Implementation Details	8
Chapter 7: Conclusion	9
References	10

1. Introduction

In today's era of rapid technological advancement, artificial intelligence and machine learning have emerged as transformative fields with applications spanning virtually every domain of human life. One of the most visually impactful and practically useful branches of AI is computer vision, which enables machines to interpret and understand the visual world. Object detection, a fundamental task in computer vision, allows a system to automatically identify and locate objects within a given image or video.

The YOLO (You Only Look Once) family of object detection algorithms has revolutionized the way real-time detection is performed. Unlike traditional region proposal based methods that require multiple passes through a network, YOLO treats object detection as a single regression problem, simultaneously predicting class labels and bounding box coordinates from full images in a single evaluation. This approach makes it extremely fast while maintaining good accuracy, making it highly suitable for real-world, real-time deployment scenarios.

YOLOv8, developed by Ultralytics, is the eighth and most recent generation of the YOLO architecture. It introduces a redesigned backbone, improved anchor-free detection head, and a cleaner, more modular codebase that is easy to use for beginners. The Nano variant (yolov8n) is specifically optimized for deployment on resource-constrained devices such as laptops, embedded systems, and edge devices, making it an ideal choice for academic projects and first-year students.

This case study documents the end-to-end development of a classroom object recognition system using YOLOv8. The system was designed to detect commonly found objects in a college or school environment such as water bottles, chairs, books, laptops, and cups. The trained model was subsequently deployed for live real-time inference using a standard webcam, demonstrating the practical feasibility of applying deep learning in everyday surroundings.

2. Problem Statement and Objectives

2.1 Problem Statement

With the increasing integration of AI-based systems in educational and workplace environments, there is a growing need for lightweight, locally deployable object recognition systems that do not depend on cloud infrastructure. Existing solutions either require powerful GPU servers, paid cloud APIs, or highly specific and expensive hardware setups, which makes them inaccessible for students and small institutions.

The problem addressed in this project is to build a simple yet functional AI-based object detection system that can be trained and run entirely on a standard student laptop using free, open-source tools. The system should be able to recognize common classroom and daily-use objects from a live webcam feed in real time.

2.2 Objectives

The following specific objectives were defined for this project:

- To collect or download a properly labeled dataset of classroom objects in YOLO format
- To configure a data.yaml file specifying the dataset structure, class names, and count
- To train the YOLOv8 Nano model for 10 epochs on the local system using CPU or GPU
- To evaluate the trained model using standard metrics such as precision, recall, and mAP
- To save the best-performing model checkpoint and use it for real-time webcam inference
- To document the complete machine learning pipeline for academic submission

2.3 Scope of the Project

The scope of this project is limited to detecting five classes of objects: bottle, chair, book, laptop, and cup. The system is designed to run on a Windows or Linux laptop with Python 3.10 or 3.11 installed. The training is intended to be performed locally without requiring internet access after initial setup. The inference is performed using a standard USB or built-in laptop webcam.

3. History of Object Detection

3.1 History of Object Detection

The field of object detection has evolved significantly over the past two decades. Early approaches relied on handcrafted features such as Histogram of Oriented Gradients (HOG) and Scale-Invariant Feature Transform (SIFT), combined with classical machine learning classifiers like Support Vector Machines (SVMs). These methods, though pioneering, were computationally expensive and limited in their ability to generalize across diverse visual conditions.

The introduction of deep learning, particularly Convolutional Neural Networks (CNNs), transformed the field. The R-CNN family (Region-based CNN, Fast R-CNN, Faster R-CNN) achieved significant accuracy improvements by using deep features, but these methods remained relatively slow due to their two-stage detection pipeline. Single-shot detectors like SSD (Single Shot MultiBox Detector) and YOLO were subsequently introduced to achieve faster inference speeds suitable for real-time applications.

3.2 Evolution of YOLO

The original YOLO model was introduced by Redmon et al. in 2015 and presented a fundamentally new approach to detection. Rather than examining potential object regions separately, YOLO divides the input image into a grid and simultaneously predicts bounding boxes and class probabilities for all grid cells in a single forward pass. This approach dramatically improved inference speed, enabling real-time detection on standard hardware.

Subsequent versions (YOLOv2, YOLOv3, YOLOv4, YOLOv5) progressively improved accuracy and speed by refining the architecture, loss functions, and data augmentation strategies. YOLOv8, released by Ultralytics in January 2023, represents the current state-of-the-art in this family, introducing an anchor-free detection head and improved training configurations that make it accessible and highly performant even for beginners.

3.3 OpenCV for Real-Time Vision

OpenCV (Open Source Computer Vision Library) is the world's most widely used open-source computer vision library. Originally developed by Intel, it provides a comprehensive suite of tools for image processing, video capture, feature detection, and machine learning integration. In this project, OpenCV is used to capture live frames from the webcam and to display the annotated output with bounding boxes and class labels in real time.

4. Dataset Description and Preprocessing

4.1 Dataset Source

The dataset used in this project was downloaded from Roboflow, an online platform that hosts publicly available annotated datasets in various formats. The selected dataset contained images of common classroom and office objects including bottles, chairs, books, laptops, and cups. Roboflow allows export in YOLO format directly, which eliminates the need for manual conversion of annotation formats.

4.2 Dataset Format

The dataset follows the standard YOLO object detection format. Each image in the dataset has a corresponding text annotation file with the same base filename. The annotation file contains one row per object instance present in the image. Each row specifies the class ID followed by the normalized bounding box coordinates in the format:

```
Class_id   x_center   y_center   width   height
```

All coordinate values are normalized to the range [0, 1] relative to the image dimensions. This normalization ensures that annotations remain valid regardless of the actual resolution of the image file.

4.3 Dataset Structure

The dataset was organized into the following directory structure as required by YOLOv8:

- data/images/train/ — Training images
- data/images/val/ — Validation images
- data/images/test/ — Test images (optional)
- data/labels/train/ — Corresponding training annotation files
- data/labels/val/ — Corresponding validation annotation files
- data/labels/test/ — Corresponding test annotation files
- data/data.yaml — Configuration file defining paths, class count, and class names

4.4 data.yaml Configuration

The data.yaml file is a critical configuration file that tells the YOLOv8 training script where to find the data and what classes to detect. The following is the data.yaml used in this project:

```
path: D:/AI_2ND/data (on Personal Desktop)
train: images/train
val: images/val
nc: 5
names: [bottle, chair, book, laptop, cup, cell_phone, human,
cookies, watch]
```

4.5 Preprocessing Steps

Before training, the following preprocessing checks were performed:

1. Verified that every image in the train, val, and test folders had a corresponding .txt label file with the same base filename.
2. Ensured no label files had incorrect class IDs beyond the defined nc range.
3. Confirmed that all bounding box coordinates were normalized and within [0, 1] range.
4. Verified the data.yaml file contained correct absolute paths suitable for the local machine.
5. Removed any corrupted or unreadable images from the dataset.

5. Model Architecture and Methodology

5.1 Why YOLOv8?

YOLOv8 was selected for this project for several compelling reasons. It is the most modern, well-maintained, and beginner-friendly object detection framework currently available. It is developed and actively maintained by Ultralytics and provides a single unified Python API for training, evaluation, and inference. The framework supports multiple model sizes ranging from Nano to Extra-Large, allowing users to balance speed and accuracy according to their hardware constraints.

5.2 Why YOLOv8 Nano?

Among the available YOLOv8 model variants (Nano, Small, Medium, Large, Extra-Large), the Nano variant was specifically selected for this project. YOLOv8n (Nano) has approximately 3.2 million parameters, making it the lightest and fastest model in the family. It is specifically engineered for deployment on low-resource devices such as mobile phones, Raspberry Pi boards, and standard student laptops. It can be trained efficiently on a CPU without requiring an NVIDIA GPU, though GPU training is supported and significantly faster when available.

5.3 Model Architecture Overview

YOLOv8 uses a CSP (Cross Stage Partial) Darknet-based backbone for feature extraction, a PANet (Path Aggregation Network) neck for feature pyramid construction, and a decoupled detection head that separately predicts objectness, class probabilities, and bounding box regression. Unlike earlier YOLO versions that used anchor-based detection, YOLOv8 uses an anchor-free approach that simplifies the training process and improves generalization.

5.4 Loss Functions

During training, YOLOv8 minimizes three types of loss:

- Bounding Box Regression Loss (Box Loss): Measures the discrepancy between predicted and ground-truth bounding box coordinates. YOLOv8 uses a variant of CIoU (Complete Intersection over Union) loss for this purpose.
- Classification Loss (Cls Loss): Cross-entropy loss that measures how well the model predicts the correct class label for each detected object.
- Distribution Focal Loss (DFL): An additional regression loss introduced in newer YOLO versions that helps the model learn a more precise distribution of bounding box edge coordinates.

As training progresses, all three loss values are expected to decrease, indicating the model is learning to make progressively better predictions on the training and validation data.

5.5 Key Mathematical Concepts

5.5.1 Intersection over Union (IoU)

IoU is the primary metric used to measure the accuracy of bounding box predictions. It is calculated as the ratio of the area of overlap between the predicted bounding box and the ground-truth bounding box to the total area covered by both boxes combined:

$$IoU = Area\ of\ Overlap\ /\ Area\ of\ Union$$

A detection is typically considered correct if IoU is greater than a threshold of 0.5. Higher IoU values indicate better spatial localization of the detected object.

5.5.2 Confidence Score

For each bounding box predicted by the model, a confidence score is assigned. This score represents the model's certainty that the region contains a valid object AND that the predicted class is correct. In the detection script, a confidence threshold of 0.5 was used, meaning only detections with a confidence score of 50% or higher were displayed on screen.

6. Implementation Details

6.1 Software Environment

Component	Details
Operating System	Windows 10/11
Programming Language	Python 3.10 / 3.11
Deep Learning Framework	Ultralytics YOLOv8 (ultralytics library with support of CUDA for gpu)
Computer Vision Library	OpenCV (opencv-python)
Dataset Format	YOLO Object Detection Format
Model Used	YOLOv8 Medium (yolov8m.pt)
Training Epochs	10
Image Size	640 x 640 pixels
Batch Size	8
Device	CPU (or CUDA GPU if available)

6.2 Project Directory Structure

The project files were organized as follows:

- AI_2ND/ — Root project directory
- AI_2ND/data/ — Contains all dataset folders and data.yaml
- AI_2ND/data/images/train|val|test/ — Image files
- AI_2ND/data/labels/train|val|test/ — Label text files
- AI_2ND/train.py — Script for training the model
- AI_2ND/detect.py — Script for real-time webcam detection
- AI_2ND/test_image.py — Script for single image inference
- AI_2ND/runs/detect/train/weights/best.pt — Best trained model

6.3 Training Script (train.py)

The training was performed using the following Python script. The script loads the pretrained YOLOv8 Nano weights and fine-tunes them on the custom dataset for 10 epochs:

```
from ultralytics import YOLO
model = YOLO('yolov8n.pt')
model.train(
data='data/data.yaml',
epochs=10,
imgsz=640,
batch=4,
device='cpu') #or use ‘0’ for CUDA GPU
```

6.4 Webcam Detection Script (detect.py)

After training, the following script was used to run real-time detection via the laptop webcam. The model loads best.pt and applies it frame-by-frame to the live webcam feed:

```
import cv2 from ultralytics import YOLO
model = YOLO('runs/detect/train/weights/best.pt')
cap = cv2.VideoCapture(0)
while True:
      ret, frame = cap.read()
      results = model(frame, conf=0.5)
      cv2.imshow('Detection', results[0].plot())
      if cv2.waitKey(1) & 0xFF == ord('q'): break
```

7. Conclusion

This case study successfully demonstrated the complete machine learning pipeline for a real-time object detection application using YOLOv8 and OpenCV. Starting from the collection and preparation of a YOLO-formatted dataset, the project covered all essential stages of an AI/ML project workflow including data preprocessing, model selection, local training, quantitative evaluation, and real-time deployment via webcam inference.

The YOLOv8 Nano model proved to be an appropriate and practical choice for this beginner-level project. Its lightweight architecture allowed training to be completed on a standard student laptop without the need for specialized GPU hardware or cloud computing services. The resulting best.pt model achieved satisfactory detection performance across all five target classes after training for just 10 epochs, demonstrating that even brief training runs can yield usable models for simple recognition tasks.

Through the development of this project, the following concepts were practically understood and applied: the structure and format of YOLO datasets, the role of loss functions in deep learning optimization, the meaning and interpretation of evaluation metrics such as precision, recall, and mAP, the mathematical foundation of IoU for bounding box evaluation, and the integration of a trained model with a live video feed using OpenCV.

The project confirms that modern AI tools have become sufficiently accessible that first-year undergraduate students can design and implement functional real-world AI applications using only free, open-source resources and commodity hardware.

References

1. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 779-788.
2. Jocher, G., Chaurasia, A., & Qiu, J. (2023). YOLO by Ultralytics (Version 8.0.0). Ultralytics. Available at: https://github.com/ultralytics/ultralytics
3. OpenCV Development Team. (2023). OpenCV: Open Source Computer Vision Library. Available at: https://opencv.org
4. Roboflow, Inc. (2023). Roboflow Dataset Hub. Available at: https://roboflow.com
5. Bochkovskiy, A., Wang, C. Y., & Liao, H. Y. M. (2020). YOLOv4: Optimal Speed and Accuracy of Object Detection. arXiv preprint arXiv:2004.10934.
6. Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.
7. Ultralytics Documentation. (2024). YOLOv8 Docs. Available at: https://docs.ultralytics.com